AWS CLOUD& DATA ENGINEERING

M. PAVAN(23BTLEI001)

Name – M. Pavan

Roll number – 23btlei001

College name- jain deemed to be university

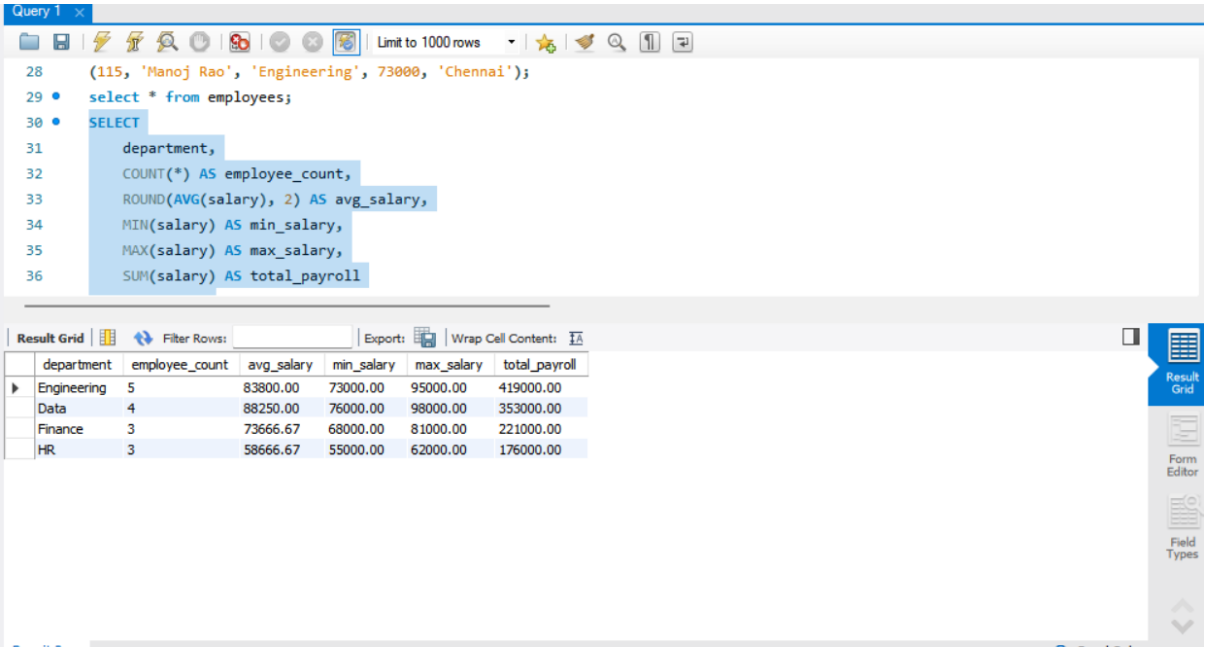
Batch 2023-2027

Date 10/10/2026

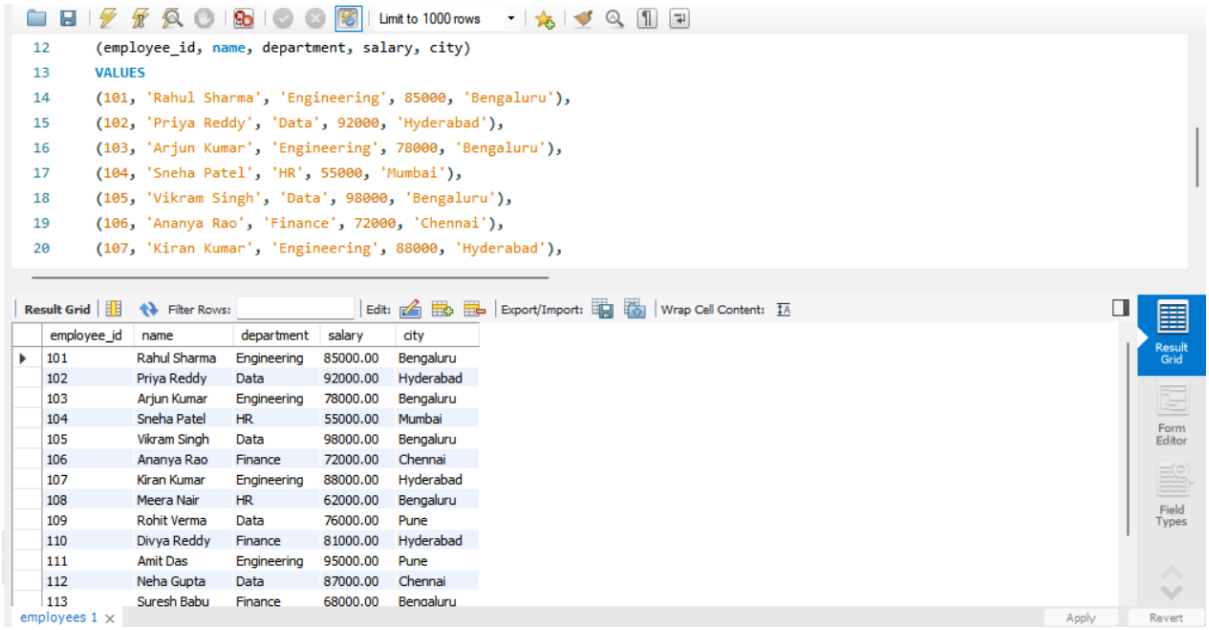
Title AWS Cloud Data Engineering

SQL Practise

SQL Queries Performed



Minimum 15 Queries



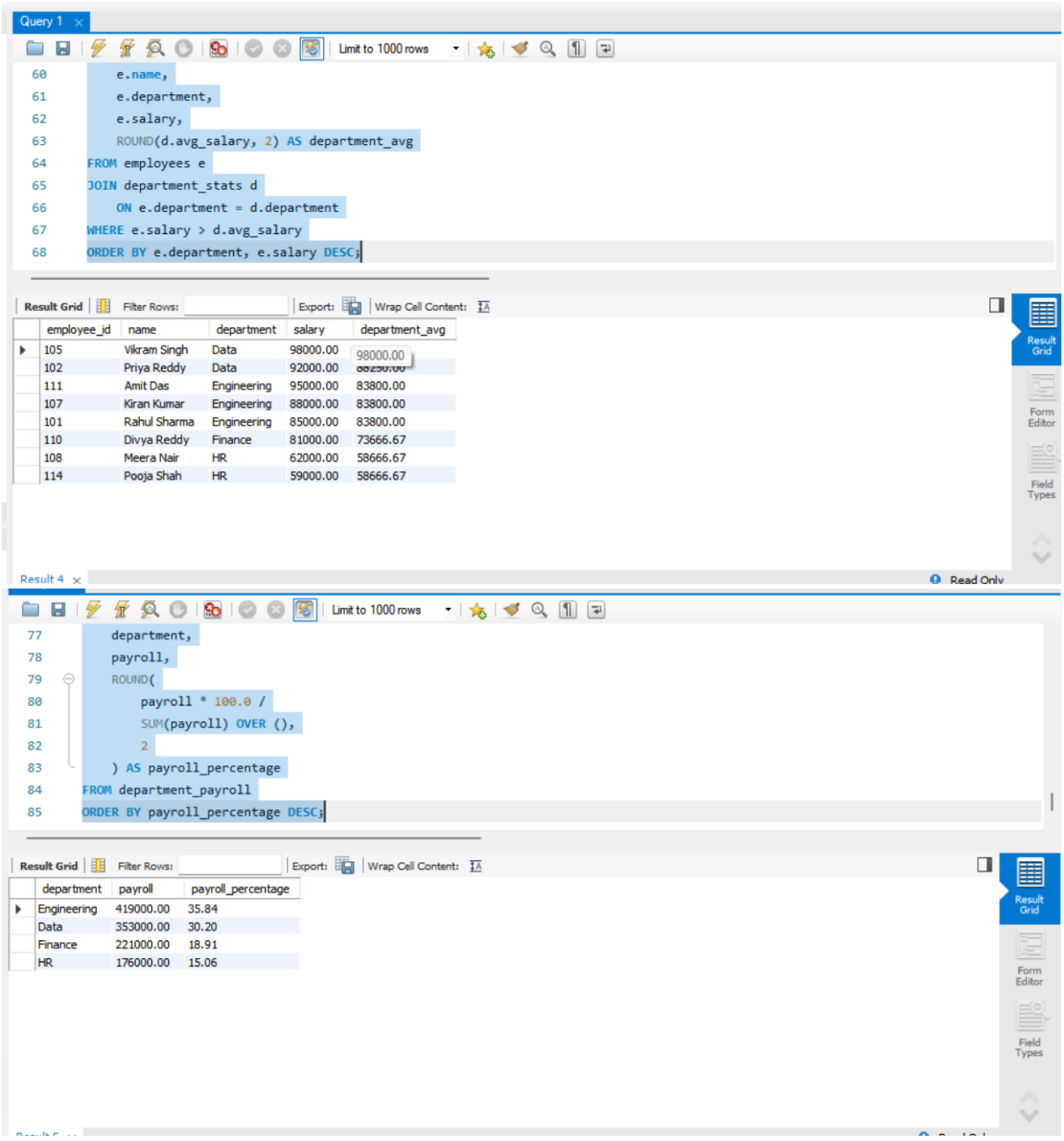
```
Limit to 1000 rows
12      (employee_id, name, department, salary, city)
13      VALUES
14      (101, 'Rahul Sharma', 'Engineering', 85000, 'Bengaluru'),
15      (102, 'Priya Reddy', 'Data', 92000, 'Hyderabad'),
16      (103, 'Arjun Kumar', 'Engineering', 78000, 'Bengaluru'),
17      (104, 'Sneha Patel', 'HR', 55000, 'Mumbai'),
18      (105, 'Vikram Singh', 'Data', 98000, 'Bengaluru'),
19      (106, 'Ananya Rao', 'Finance', 72000, 'Chennai'),
20      (107, 'Kiran Kumar', 'Engineering', 88000, 'Hyderabad'),
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_id	name	department	salary	city
101	Rahul Sharma	Engineering	85000.00	Bengaluru
102	Priya Reddy	Data	92000.00	Hyderabad
103	Arjun Kumar	Engineering	78000.00	Bengaluru
104	Sneha Patel	HR	55000.00	Mumbai
105	Vikram Singh	Data	98000.00	Bengaluru
106	Ananya Rao	Finance	72000.00	Chennai
107	Kiran Kumar	Engineering	88000.00	Hyderabad
108	Meera Nair	HR	62000.00	Bengaluru
109	Rohit Verma	Data	76000.00	Pune
110	Divya Reddy	Finance	81000.00	Hyderabad
111	Amit Das	Engineering	95000.00	Pune
112	Neha Gupta	Data	87000.00	Chennai
113	Suresh Babu	Finance	68000.00	Bengaluru

Result Grid | Form Editor | Field Types

employees 1 | Apply | Revert


Query 1
Limit to 1000 rows
e.name,
e.department,
e.salary,
ROUND(d.avg_salary, 2) AS department_avg
FROM employees e
JOIN department_stats d
ON e.department = d.department
WHERE e.salary > d.avg_salary
ORDER BY e.department, e.salary DESC;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
employee_id name department salary department_avg
105 Vikram Singh Data 98000.00 98000.00
102 Priya Reddy Data 92000.00
111 Amit Das Engineering 95000.00 83800.00
107 Kiran Kumar Engineering 88000.00 83800.00
101 Rahul Sharma Engineering 85000.00 83800.00
110 Divya Reddy Finance 81000.00 73666.67
108 Meera Nair HR 62000.00 58666.67
114 Pooja Shah HR 59000.00 58666.67
Result Grid
Form Editor
Field Types
Result 4
Read Only

Limit to 1000 rows
department,
payroll,
ROUND(
payroll * 100.0 /
SUM(payroll) OVER (),
2
) AS payroll_percentage
FROM department_payroll
ORDER BY payroll_percentage DESC;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
department payroll payroll_percentage
Engineering 419000.00 35.84
Data 353000.00 30.20
Finance 221000.00 18.91
HR 176000.00 15.06
Result Grid
Form Editor
Field Types

Query 1

```
97  )
98  SELECT
99      employee_id,
100     name,
101     department,
102     salary
103 FROM ranked_employees
104 WHERE salary_rank = 1
105 ORDER BY salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	name	department	salary
105	Vikram Singh	Data	98000.00
111	Amit Das	Engineering	95000.00
110	Divya Reddy	Finance	81000.00
108	Meera Nair	HR	62000.00

Result 6 | Read Only

Query 1

```
112         salary - (SELECT AVG(salary) FROM employees),
113         2
114     ) AS difference_from_company_avg
115 FROM employees
116 WHERE salary > (
117     SELECT AVG(salary)
118     FROM employees
119 )
120 ORDER BY difference_from_company_avg DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	name	department	salary	difference_from_company_avg
105	Vikram Singh	Data	98000.00	20066.67
111	Amit Das	Engineering	95000.00	17066.67
102	Priya Reddy	Data	92000.00	14066.67
107	Kiran Kumar	Engineering	88000.00	10066.67
112	Neha Gupta	Data	87000.00	9066.67
101	Rahul Sharma	Engineering	85000.00	7066.67
110	Divya Reddy	Finance	81000.00	3066.67
103	Arjun Kumar	Engineering	78000.00	66.67

Query 1

```
120  ORDER BY difference_from_company_avg DESC;
121  SELECT
122      city,
123      COUNT(*) AS employee_count,
124      SUM(salary) AS city_payroll,
125      ROUND(AVG(salary), 2) AS avg_salary
126  FROM employees
127  GROUP BY city
128  ORDER BY city_payroll DESC;
```

Result Grid

city	employee_count	city_payroll	avg_salary
Bengaluru	5	391000.00	78200.00
Hyderabad	4	320000.00	80000.00
Chennai	3	232000.00	77333.33
Pune	2	171000.00	85500.00
Mumbai	1	55000.00	55000.00

Result 8 — Read Only

Query 1

```
133      department,
134      salary,
135      CASE
136          WHEN salary >= 90000 THEN 'High'
137          WHEN salary >= 75000 THEN 'Medium'
138          ELSE 'Entry'
139      END AS salary_band
140  FROM employees
141  ORDER BY salary DESC;
```

Result Grid

employee_id	name	department	salary	salary_band
105	Vikram Singh	Data	98000.00	High
111	Amit Das	Engineering	95000.00	High
102	Priya Reddy	Data	92000.00	High
107	Kiran Kumar	Engineering	88000.00	Medium
112	Neha Gupta	Data	87000.00	Medium
101	Rahul Sharma	Engineering	85000.00	Medium
110	Divya Reddy	Finance	81000.00	Medium
103	Arjun Kumar	Engineering	78000.00	Medium
109	Rohit Verma	Data	76000.00	Medium
115	Manoj Rao	Engineering	73000.00	Entry
106	Ananya Rao	Finance	72000.00	Entry
113	Suresh Babu	Finance	68000.00	Entry
108	Meera Nair	HR	62000.00	Entry

Result 9 — Read Only

Query 1

```
        ) AS salary_rank
    FROM employees
)
SELECT DISTINCT salary AS second_highest_salary
FROM salary_levels
WHERE salary_rank = 2;

WITH department_stats AS (
    SELECT
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

second_highest_salary
95000.00

Result 10 | Read Only

Query 1

```
    ) AS difference
FROM department_stats
ORDER BY difference DESC;

SELECT
    department,
    COUNT(*) AS employee_count,
    ROUND(AVG(salary), 2) AS avg_salary
FROM employees
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department	department_avg	company_avg	difference
Data	88250.00	77933.33	10316.67
Engineering	83800.00	77933.33	5866.67
Finance	73666.67	77933.33	-4266.67
HR	58666.67	77933.33	-19266.67

Result 11 | Read Only

Query 1

```
181     FROM employees
182     GROUP BY department
183     HAVING COUNT(*) >= 3
184        AND AVG(salary) > 75000
185     ORDER BY avg_salary DESC;
186
187 •   WITH ranked_employees AS (
188         SELECT
189             employee_id,
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department	employee_count	avg_salary
Data	4	88250.00
Engineering	5	83800.00

Result 12 | Read Only

Query 1

```
206     WHERE row_num <= 2
207     ORDER BY department, row_num;
208
209 •   SELECT
210         employee_id,
211         name,
212         department,
213         salary,
214         SUM(salary) OVER (
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	name	department	salary	row_num
105	Vikram Singh	Data	98000.00	1
102	Priya Reddy	Data	92000.00	2
111	Amit Das	Engineering	95000.00	1
107	Kiran Kumar	Engineering	88000.00	2
110	Divya Reddy	Finance	81000.00	1
106	Ananya Rao	Finance	72000.00	2
108	Meera Nair	HR	62000.00	1
114	Pooja Shah	HR	59000.00	2

```
        PARTITION BY department
        ORDER BY salary DESC, employee_id
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW
    ) AS cumulative_department_payroll
FROM employees
ORDER BY department, salary DESC, employee_id;

SELECT
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	name	department	salary	cumulative_department_payroll
105	Vikram Singh	Data	98000.00	98000.00
102	Priya Reddy	Data	92000.00	190000.00
112	Neha Gupta	Data	87000.00	277000.00
109	Rohit Verma	Data	76000.00	353000.00
111	Amit Das	Engineering	95000.00	95000.00
107	Kiran Kumar	Engineering	88000.00	183000.00
101	Rahul Sharma	Engineering	85000.00	268000.00
103	Arjun Kumar	Engineering	78000.00	346000.00
115	Manoj Rao	Engineering	73000.00	419000.00
110	Divya Reddy	Finance	81000.00	81000.00
106	Ananya Rao	Finance	72000.00	153000.00
113	Suresh Babu	Finance	68000.00	221000.00
108	Meera Nair	HR	62000.00	62000.00

Result 16 | Read Only

Query 1

```
SELECT
    department,
    COUNT(*) AS employee_count,
    ROUND(AVG(salary), 2) AS avg_salary,
    ROUND(STDDEV_POP(salary), 2) AS salary_std_deviation
FROM employees
GROUP BY department
ORDER BY salary_std_deviation DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

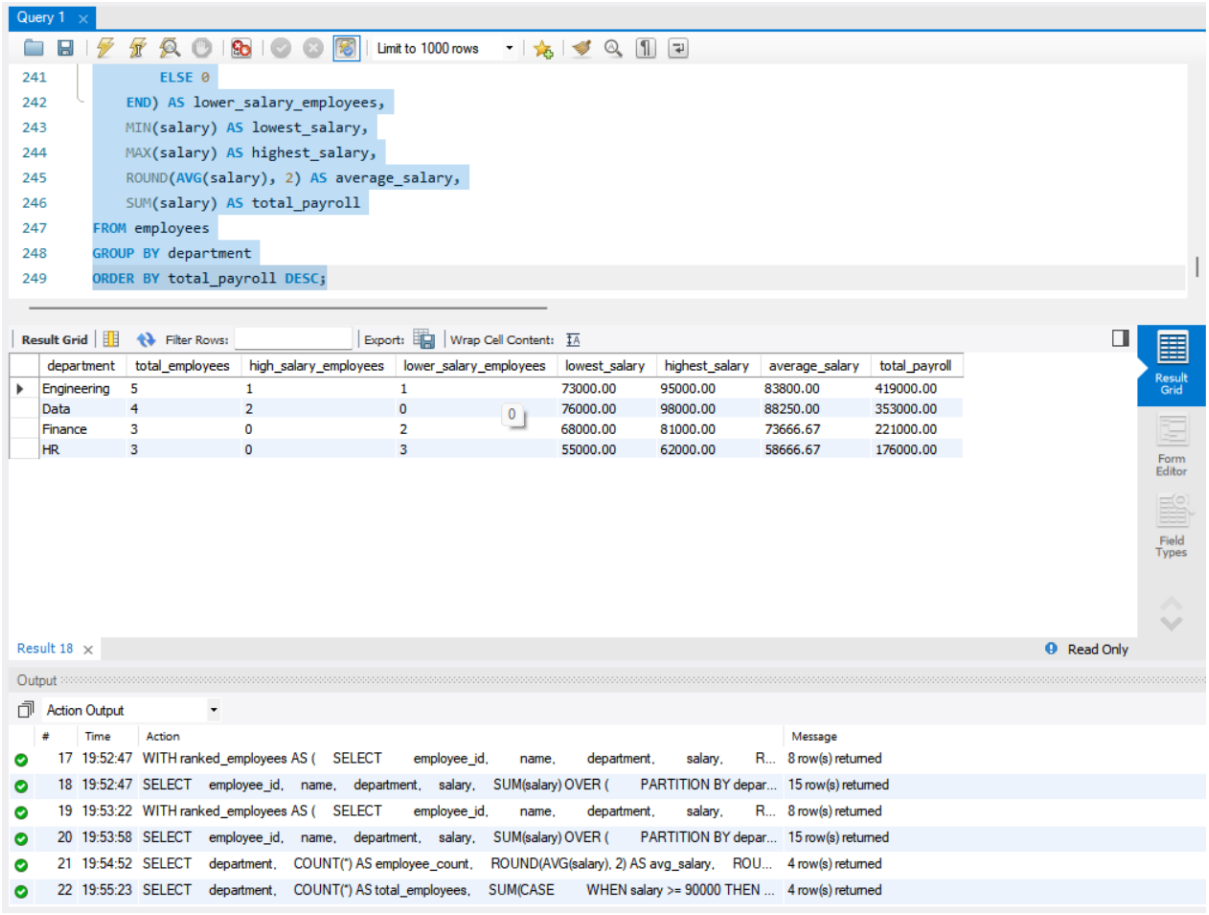
department	employee_count	avg_salary	salary_std_deviation
Data	4	88250.00	8073.88
Engineering	5	83800.00	7678.54
Finance	3	73666.67	5436.5
HR	3	58666.67	2867.44

Result 17 | Read Only

Output

Action Output

#	Time	Action	Message
16	19:51:55	SELECT department, COUNT(*) AS employee_count, ROUND(AVG(salary), 2) AS avg_salary FROM e...	2 row(s) returned
17	19:52:47	WITH ranked_employees AS (SELECT employee_id, name, department, salary, R...	8 row(s) returned
18	19:52:47	SELECT employee_id, name, department, salary, SUM(salary) OVER (PARTITION BY depar...	15 row(s) returned
19	19:53:22	WITH ranked_employees AS (SELECT employee_id, name, department, salary, R...	8 row(s) returned
20	19:53:58	SELECT employee_id, name, department, salary, SUM(salary) OVER (PARTITION BY depar...	15 row(s) returned
21	19:54:52	SELECT department, COUNT(*) AS employee_count, ROUND(AVG(salary), 2) AS avg_salary, ROU...	4 row(s) returned

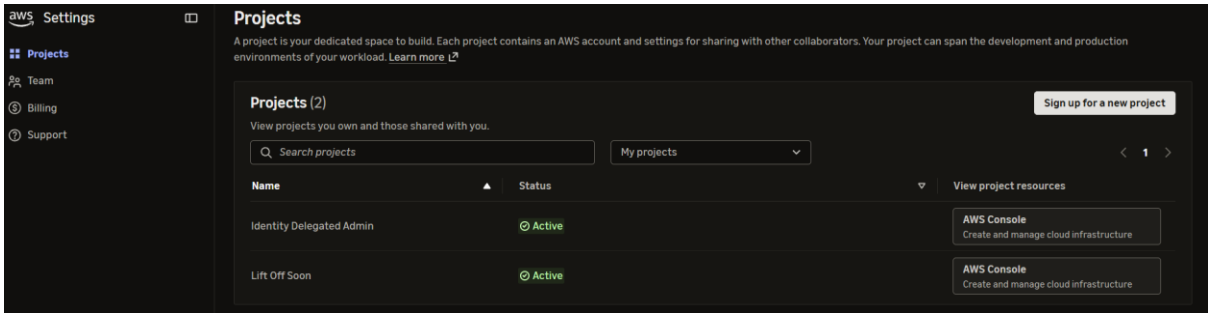


```
        ELSE 0
    END) AS lower_salary_employees,
    MIN(salary) AS lowest_salary,
    MAX(salary) AS highest_salary,
    ROUND(AVG(salary), 2) AS average_salary,
    SUM(salary) AS total_payroll
FROM employees
GROUP BY department
ORDER BY total_payroll DESC;
```

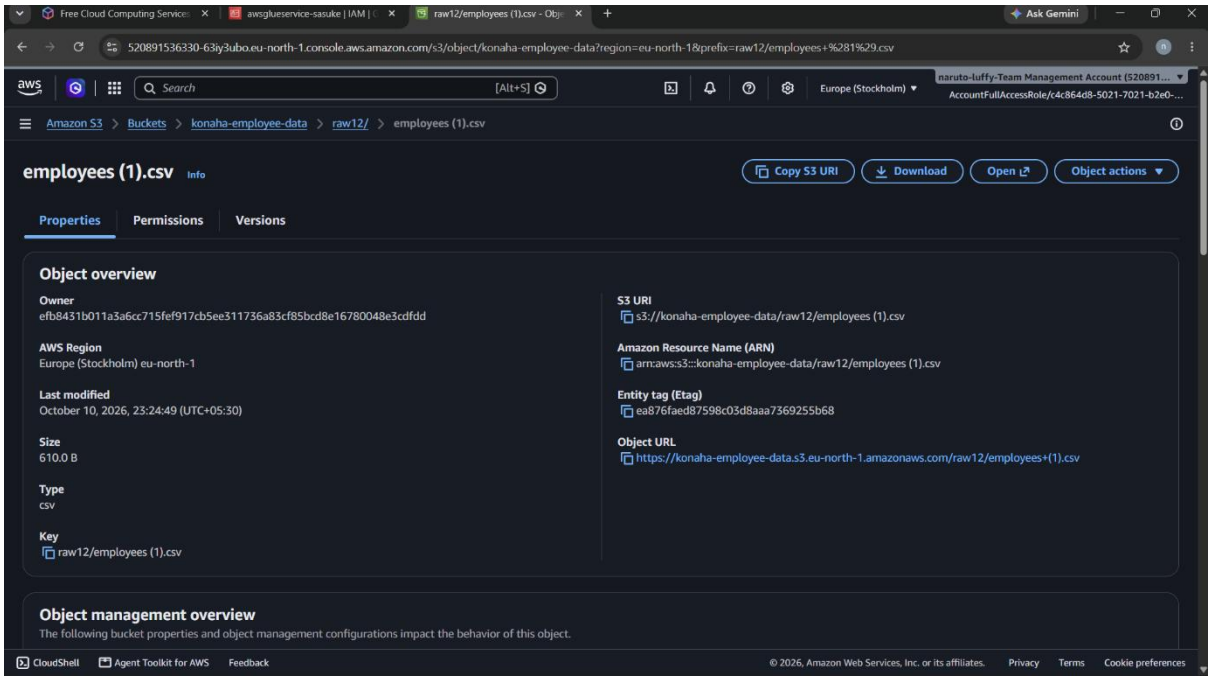
department	total_employees	high_salary_employees	lower_salary_employees	lowest_salary	highest_salary	average_salary	total_payroll
Engineering	5	1	1	73000.00	95000.00	83800.00	419000.00
Data	4	2	0	76000.00	98000.00	88250.00	353000.00
Finance	3	0	2	68000.00	81000.00	73666.67	221000.00
HR	3	0	3	55000.00	62000.00	58666.67	176000.00

#	Time	Action	Message
17	19:52:47	WITH ranked_employees AS (SELECT employee_id, name, department, salary, R...	8 row(s) returned
18	19:52:47	SELECT employee_id, name, department, salary, SUM(salary) OVER (PARTITION BY depar...	15 row(s) returned
19	19:53:22	WITH ranked_employees AS (SELECT employee_id, name, department, salary, R...	8 row(s) returned
20	19:53:58	SELECT employee_id, name, department, salary, SUM(salary) OVER (PARTITION BY depar...	15 row(s) returned
21	19:54:52	SELECT department, COUNT(*) AS employee_count, ROUND(AVG(salary), 2) AS avg_salary, ROU...	4 row(s) returned
22	19:55:23	SELECT department, COUNT(*) AS total_employees, SUM(CASE WHEN salary >= 90000 THEN ...	4 row(s) returned

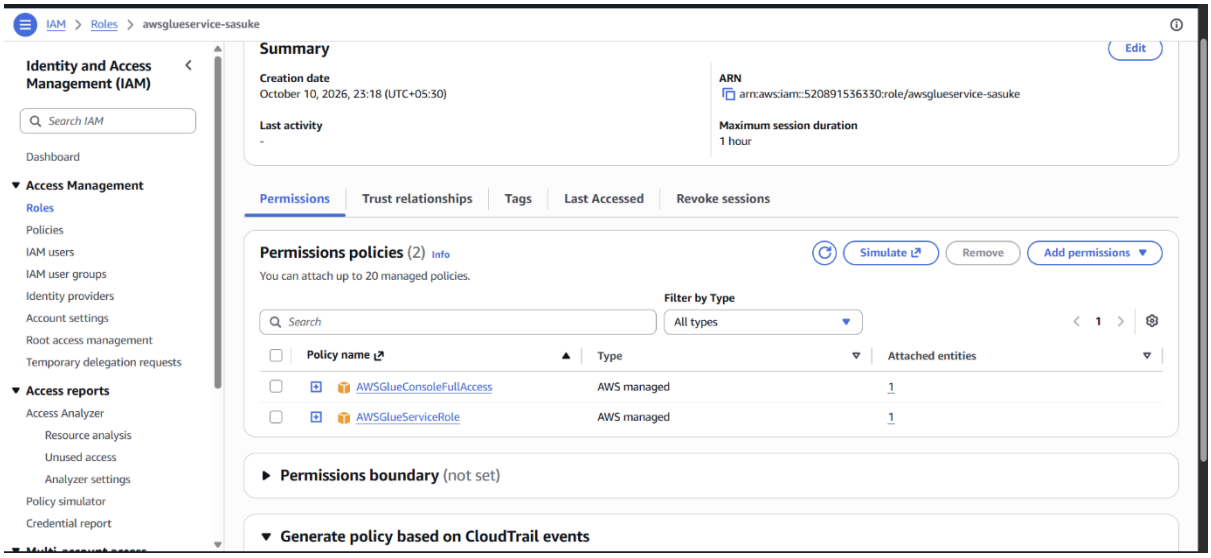
AWS Account Region



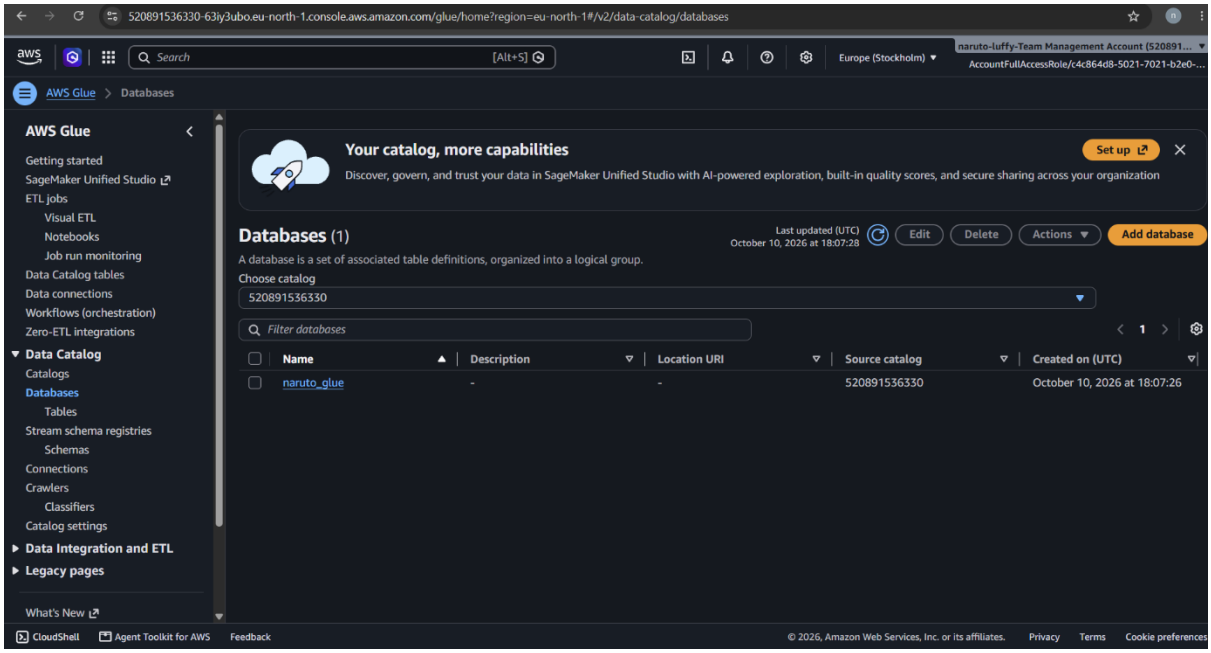
S3 Bucket created



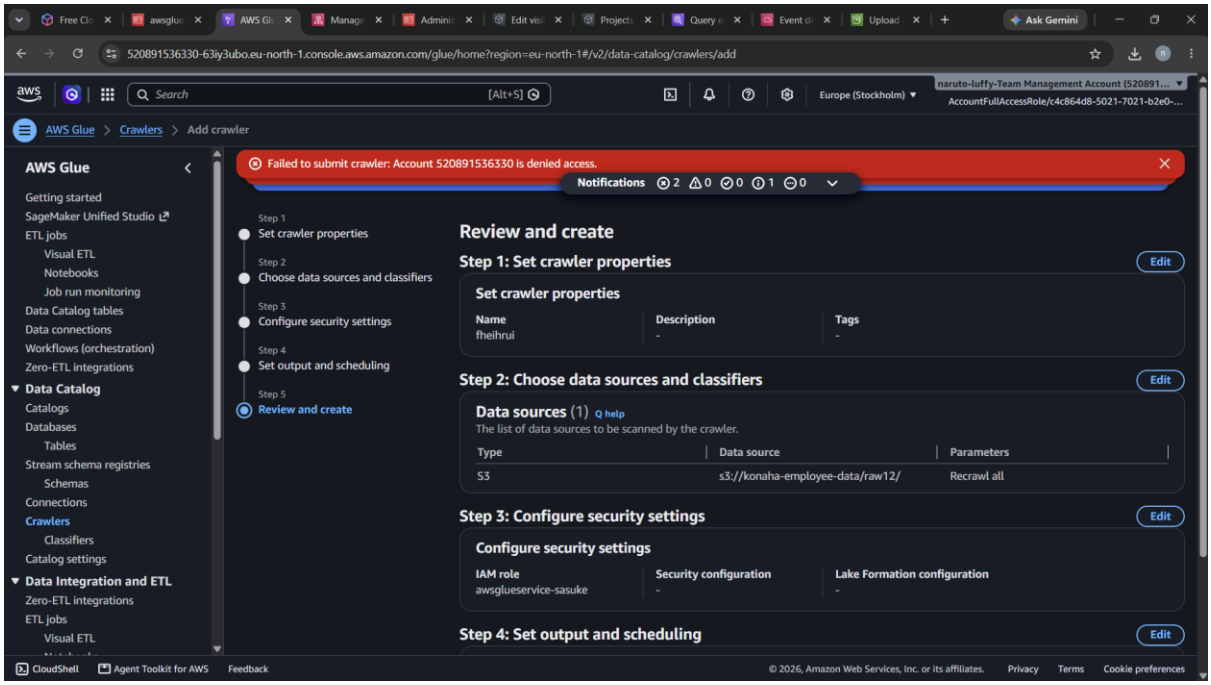
IAM



AWS Glue database

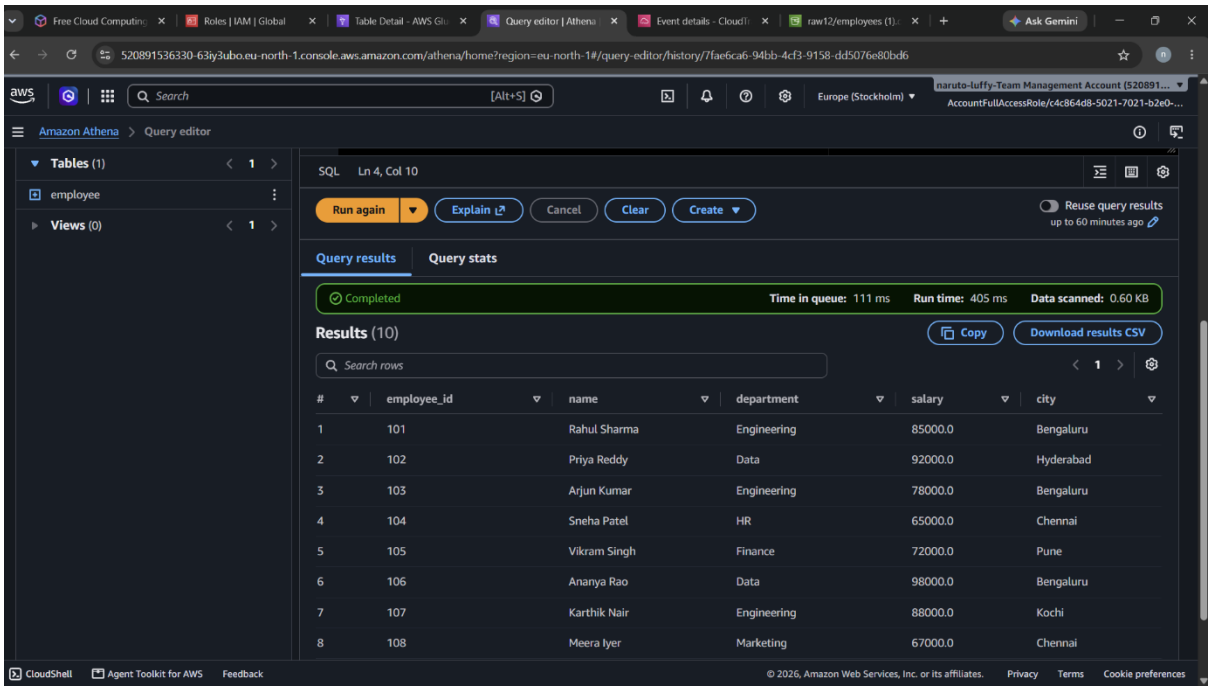


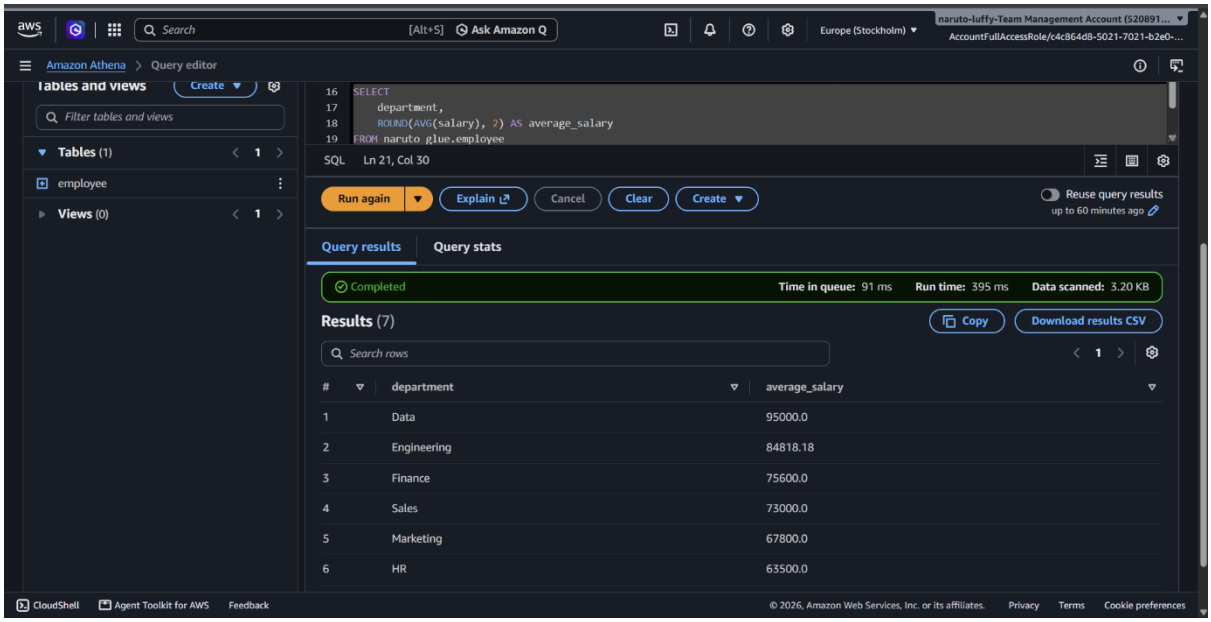
Glue crawler



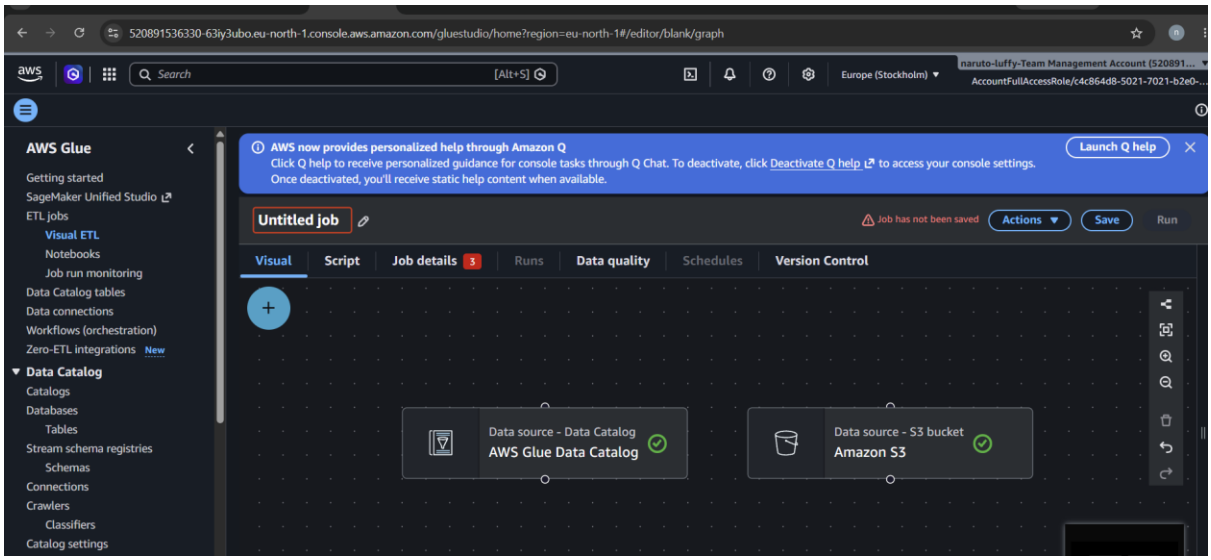
Failed to create the crawler

Athena


Amazon Athena > Query editor
Tables (1)
employee
Views (0)
SQL Ln 4, Col 10
Run again
Explain
Cancel
Clear
Create
Reuse query results
up to 60 minutes ago
Query results
Query stats
Completed
Time in queue: 111 ms
Run time: 405 ms
Data scanned: 0.60 KB
Results (10)
Copy
Download results CSV
Search rows
employee_id name department salary city
1 101 Rahul Sharma Engineering 85000.0 Bengaluru
2 102 Priya Reddy Data 92000.0 Hyderabad
3 103 Arjun Kumar Engineering 78000.0 Bengaluru
4 104 Sneha Patel HR 65000.0 Chennai
5 105 Vikram Singh Finance 72000.0 Pune
6 106 Ananya Rao Data 98000.0 Bengaluru
7 107 Karthik Nair Engineering 88000.0 Kochi
8 108 Meera Iyer Marketing 67000.0 Chennai
CloudShell
Agent Toolkit for AWS
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Amazon Athena > Query editor
Tables and views
Create
Filter tables and views
Tables (1)
employee
Views (0)
SELECT
department,
ROUND(AVG(salary), 2) AS average_salary
FROM naruto_glue.employee
SQL Ln 21, Col 30
Run again
Explain
Cancel
Clear
Create
Reuse query results
up to 60 minutes ago
Query results
Query stats
Completed
Time in queue: 91 ms
Run time: 395 ms
Data scanned: 3.20 KB
Results (7)
Copy
Download results CSV
Search rows
department average_salary
1 Data 95000.0
2 Engineering 84818.18
3 Finance 75600.0
4 Sales 73000.0
5 Marketing 67800.0
6 HR 63500.0
CloudShell
Agent Toolkit for AWS
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Failed to create the visual ETL



Employee management data

On SQL workbench

Step 1:
open Aws Console account

→ Create S3 bucket
→ upload the csv file.
create the folder
→ Add files
click upload.

* Create Aws Glue database
open Data catalog
Database Create

Next Aws Glue table
create table

Next create IAM role
with the full service role

* open Amazon Athena
Athena Query editor
select the database
Select the table

Excute the tables

Final architecture

